Control de Concurrència: granularitat múltiple i deadlocks

Gestió i Administració de Bases de Dades. Grau en Enginyeria Informàtica

Oriol Ramos Terrades

Luis Santiago Luévano

Departament de Ciències de la Computació

Escola d'Enginyeria UAB

DC UAB

Continguts

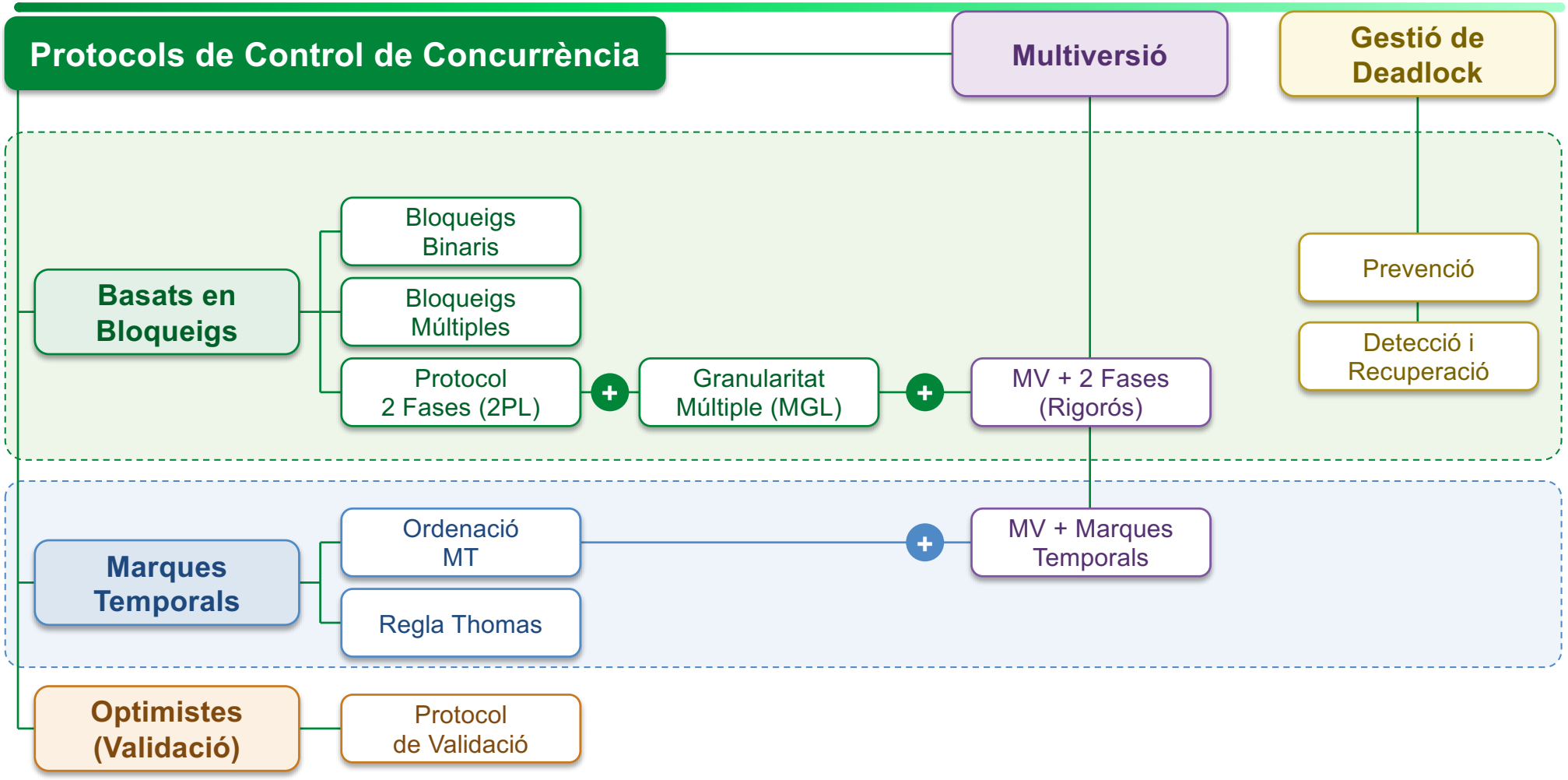
01 Introducció

02 **Protocols basats en bloqueigs**

03 Gestió de deadlocks

04 Altres problemes de concurrència

Classificació dels Protocols de Control de Concurrència (PCC)



Protocols basats en bloqueigs

01 Bloqueigs binaris i múltiples

02 Protocols de 2 Fases

03 **Protocols de Granularitat Múltiple**

Protocols basats en granularitat múltiple

Fins ara, cada element de dades es tractava individualment. A vegades convé **agrupar elements** i tractar-los com una unitat:

Ti vol bloquejar tota la BD	Ti vol bloquejar certs elements
No cal bloquejar cada element per separat. **Evita overhead**	No cal bloquejar tota la BD. **Manté la concurrència**

→ Concepte de **granularitat**: la mida de l'element que es bloqueja.

Granularitat múltiple

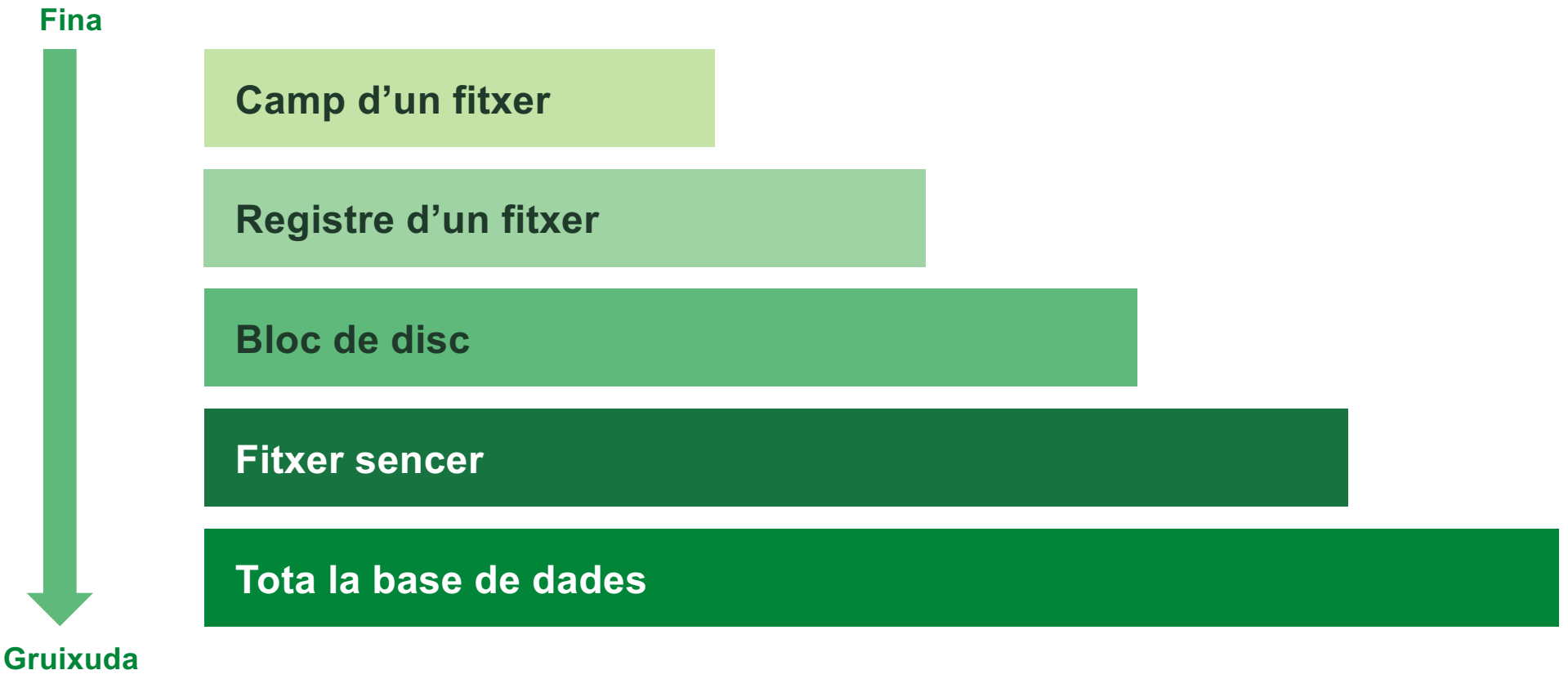
Granularitat	**Mida de la dada a controlar**: des d'un camp de fitxer fins a un fitxer sencer o una unitat de disc.

La mida és un factor que afecta el control de concurrència:

Mida petita	Mida gran
• Permet **més concurrència**. • Hi pot haver **més conflictes** entre transaccions.	• Permet **menys concurrència**. • **Evita conflictes** entre transaccions.

Granularitat múltiple

La BD es veu com un conjunt d'elements (recursos) de mida creixent:



Com més gran és l'element, **menys concurrència** i **menys overhead**.

Granularitat múltiple

La granularitat triada afecta el rendiment:

Granularitat gruixuda

Mides grans

- Grau de concurrència baix: menys transaccions concurrents.
- Pocs recursos → poques estructures per mantenir-los.
- Menys operacions de bloqueig → menys overhead.

Granularitat fina

Mides petites

- Grau de concurrència alt: més transaccions concurrents.
- Molts recursos → moltes estructures per mantenir-los.
- Més operacions de bloqueig → més overhead.

Granularitat múltiple

Quina és la granularitat òptima? Depèn de les transaccions:

T accedeix a **pocs recursos**

Granularitat fina
registre

T accedeix a **molts recursos**

Granularitat gruixuda
blocs de registres o fitxer sencer

Per combinar-les: es defineixen diversos nivells de granularitat, organitzats en un **arbre de granularitats** (jerarquia).

Granularitat múltiple

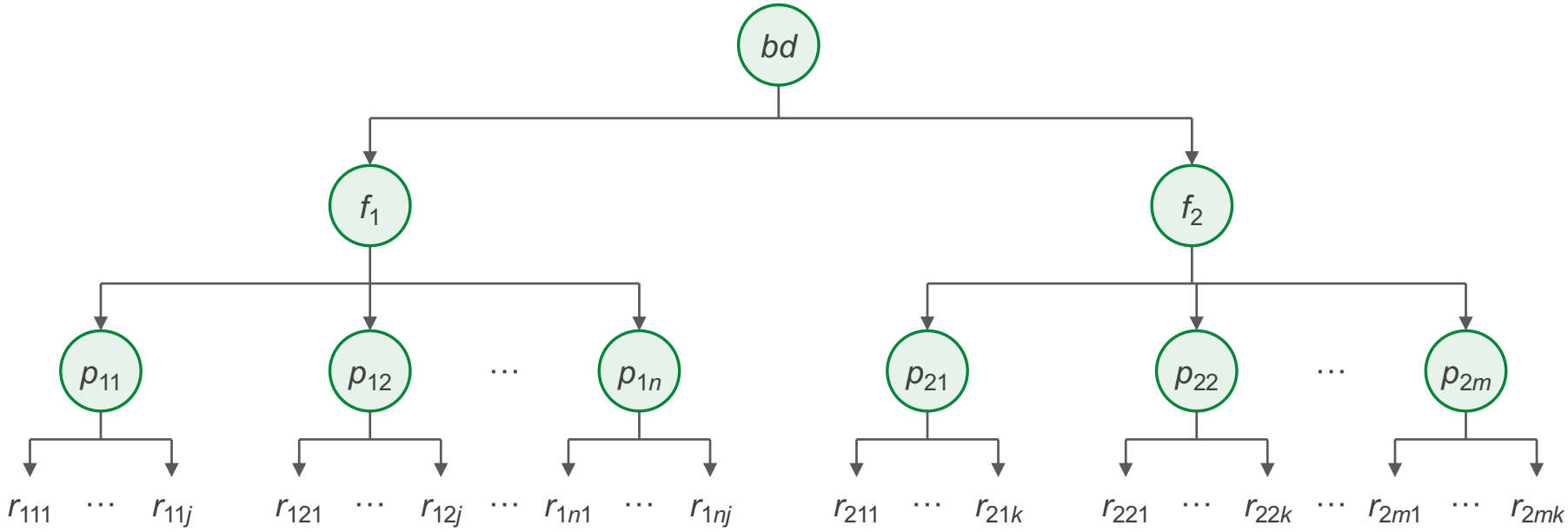
Cada node es pot bloquejar individualment, en mode compartit (**S**) o exclusiu (**X**).

Quan es bloqueja un node, els fills queden bloquejats en el mateix mode:

Bloqueig explícit	El fa la transacció sobre el node.
Bloqueig implícit	El reben els fills de l'element de dades que té el bloqueig explícit.

Granularitat múltiple

- **Arbre de granularitat**:
  - Node arrel: **Tota la BD**.
  - Cada zona té nodes (grups d'elements de dades).
  - Fulles: **registres**.



Granularitat múltiple: bloqueigs d'intenció

Un bloqueig d'intenció avisa que hi ha un bloqueig explícit més avall a l'arbre.

1 Bloqueig explícit

La transacció vol bloquejar explícitament un node Q d'un nivell inferior.

2 Intenció als ascendents

Abans, **marca tots els ascendents** del node en mode intencionat.

3 De l'arrel fins a Q

Recorre l'arbre des de l'arrel fins a Q i bloqueja cada node en mode intencionat, si pot.

Ordre de bloqueig: bd → fitxer → pàgina → registre

Granularitat múltiple

Tres tipus de bloqueigs d'intenció:

IS	IX	SIX
D'intenció compartida	D'intenció exclusiva	Compartit amb intenció exclusiva
Es demanarà un bloqueig **compartit** en algun dels nodes descendents.	Es demanarà un bloqueig **exclusiu** en algun dels descendents.	El node té bloqueig **compartit**, però algun dels descendents té bloqueig **exclusiu**.

Granularitat múltiple

Cada node *N* té en els fills diferents valors de bloqueig d'intenció, pel que *N* guarda el valor més important, que compara amb el bloqueig que demana la transacció *Ti* → **matriu de compatibilitat de bloqueigs**.

	Bloqueig que té el node *N*				
Bloqueig que demana *Ti*	**IS**	**IX**	**S**	**SIX**	**X**
IS	**Sí**	**Sí**	**Sí**	**Sí**	No
IX	**Sí**	**Sí**	No	No	No
S compartit	**Sí**	No	**Sí**	No	No
SIX	**Sí**	No	No	No	No
X exclusiu	No	No	No	No	No

Sí compatible

La transacció té accés a l'objecte.

No incompatible

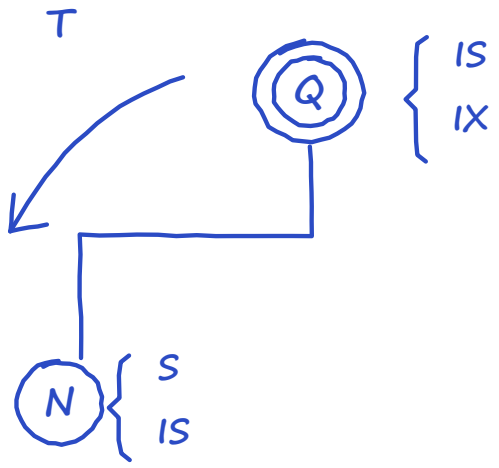
La transacció que vol accedir a l'objecte ha d'esperar.

Granularitat múltiple

Protocol **MGL** (*Multiple Granularity Locking*) amb bloqueigs d'intenció. Regles perquè `T` bloquegi un node `N`:

1	Respectar la **matriu de compatibilitat** de bloqueigs.
2	Bloquejar primer el **node arrel** de l'arbre, en qualsevol mode.
3	Bloquejar `N` en **S o IS** només si el pare `Q` ja està bloquejat per `T` en **IS o IX**.

Exemple de la regla 3

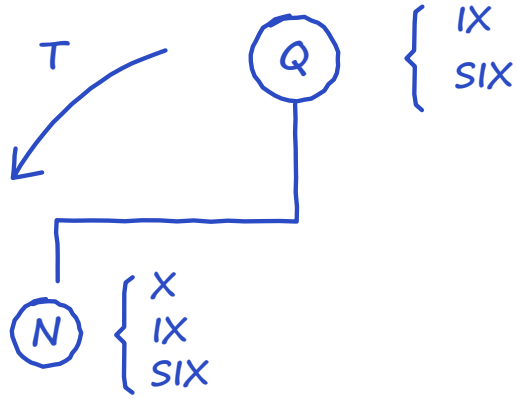


Granularitat múltiple

Regles del protocol **MGL** (continuació):

4	Bloquejar `N` en **X, IX o SIX** només si el pare `Q` ja està bloquejat per `T` en **IX o SIX**.
5	Bloquejar només si `T` encara **no ha desbloquejat cap node** (protocol 2PLP).
6	Desbloquejar `N` només si **cap fill de `N`** està bloquejat per `T`.

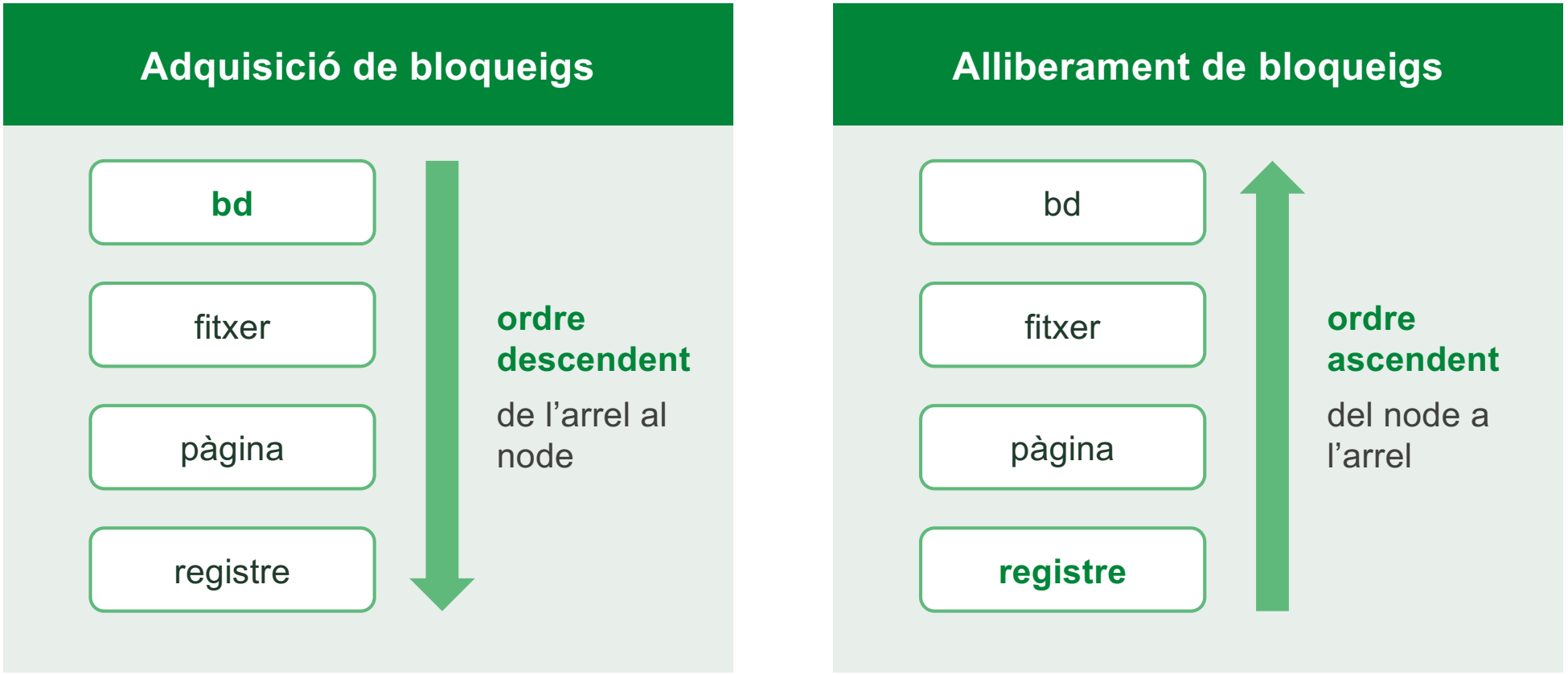
Exemple de la regla 4



Bloquejar: de l'arrel cap avall · Desbloquejar: de les fulles cap amunt

Granularitat múltiple

Observacions sobre l'ordre dels bloqueigs a l'arbre:



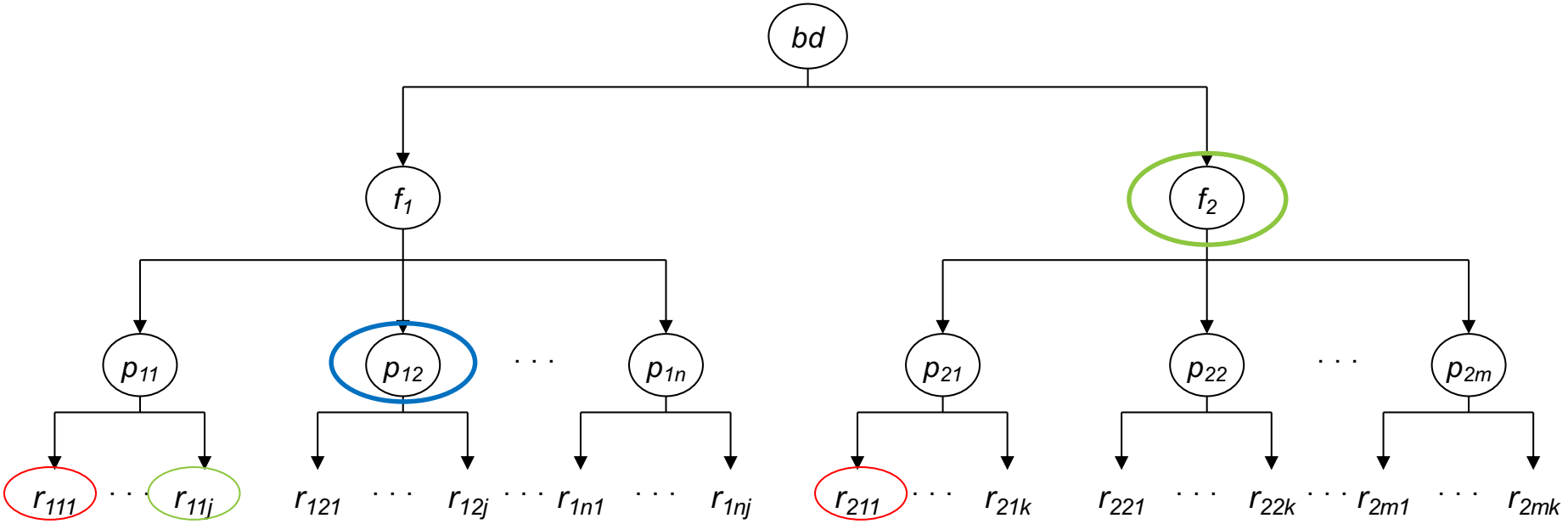
Granularitat múltiple

Exemple de 3 transaccions:

T_1
Vol actualitzar els registres r_{111} i r_{211}.

T_2
Vol actualitzar tots els registres de la pàgina p_{12}.

T_3
Vol llegir el registre r_{11j} i el fitxer f_2 sencer.



Granularitat múltiple (MGL)

El protocol MGL és adequat per a:

Barreja de transaccions	Índexs
Curtes: accedeixen a pocs elements (registres o camps). **Llargues:** accedeixen a fitxers sencers. Menys bloqueigs i menys sobrecàrrega que amb un sol nivell de granularitat.	Control de concurrència en **arbres B+ equilibrats**.

Seqüencialitat: per conflictes, perquè es basa en el 2PLP. Tal com està definit, no garanteix recuperabilitat ni evita les cancel·lacions en cascada. Per a planificacions **estrictes** (recuperables i sense cascada), cal el 2PLP estricte o el rigorós.

Limitació: no és lliure de *deadlock*. Hi ha tècniques per reduir-ne la freqüència i eliminar-los.

Continguts

01 Introducció

02 Protocols basats en bloqueigs

03 **Gestió de deadlocks**

04 Altres problemes de concurrència

Gestió de deadlocks

01 **Prevenció**

02 Detecció i recuperació

Control de concurrència: interbloqueig, prevenció, graf d'espera i recuperació

PREVENCIÓ, DETECCIÓ I RECUPERACIÓ DE *DEADLOCKS*

Escola d'Enginyeria UAB

DC UAB

Interbloqueig (*Deadlock*)

Es produeix quan una transacció T_3 està esperant algun recurs bloquejat per T_4 i aquest està esperant per algun recurs bloquejat per T3. Exemple:

PLANIFICACIÓ D						
t	*T3*	*T4*	*T3 bloqueja*	*T3 espera*	*T4 bloqueja*	*T4 espera*
1	bloqueja_x(Y)		**Y**			
2	read_item(Y)		Y			
3	Y=Y-50		Y			
4	write_item(Y)		Y			
5		bloqueja_s(X)	Y		**X**	
6		read_item(X)	Y		X	
7		bloqueja_s(Y)	Y		X	**Y**
8	bloqueja_x(X)		Y	**X**	X	Y

- T_3, T_4 **no progressaran mai**: *Deadlock*
- Pot provocar **reacció en cadena** sobre altres transaccions que vulguin X, Y i que hauran d'esperar.

Interbloqueig (*Deadlock*)

Com se'n surt d'un interbloqueig?

> **Solució:** fer retrocedir una o diverses transaccions implicades perquè alliberin els elements de dades (recursos) en conflicte.

El retrocés pot ser:

Retrocés total	Retrocés parcial
Reiniciar la transacció des del principi.	Retrocedir només fins al punt on es va obtenir el bloqueig en conflicte.

Interbloqueig (*Deadlock*)

Estratègies per afrontar el *deadlock*:

Prevenció

- Polítiques que **eviten que es produeixi**.
- Quan la probabilitat de *deadlock* és **alta**.

Detecció i recuperació

- Algorismes que **comproven periòdicament** si es produeix i, si passa, recuperen un estat segur.
- Quan la probabilitat de *deadlock* és **baixa**.
- Afegeix overhead.

Límits de temps

- Esquema **híbrid** entre prevenció i detecció.

Prevenció *Deadlock*: enfocaments

Dos enfocaments per prevenir *deadlocks*, cadascun amb els seus protocols.

1. Eliminar esperes cícliques

Els bloqueigs es fan **conjuntament**.

Protocols

a	2PLP
b	Ordenar recursos
c	Ordenar recursos amb 2PLP

2. Expropiació i retrocés

Fer **retrocedir** una transacció si el bloqueig que vol fer pot portar a *deadlock*.

Protocols

d	Esperar-Morir (*Wait-Die*)
e	Ferir-Esperar (*Wound-Wait*)

Prevenció *Deadlock*

Eliminar esperes cícliques: primers dos protocols.

a 2PLP conservador

Cada `T` **bloqueja per endavant** tots els recursos que necessita. Si no els pot aconseguir, allibera els que té i ho torna a provar més tard.

Inconvenients

- Costa predir quins recursos necessitarà abans d'executar.
- Limita la concurrència i aprofita poc els recursos.

Seqüencialitat per conflictes i lliure de deadlock. Per ser **estricte**, ha de mantenir els bloqueigs exclusius fins al commit.

b Ordenar recursos

Els recursos de la BD s'**ordenen** i cada transacció els bloqueja en aquest ordre. Evita el bloqueig cíclic `T1<T2, T2<T1`.

Inconvenients i observacions

- Limita la concurrència.
- Esquema semblant al Protocol per Arbre (PA).

Només evita el deadlock: sense 2PLP, no assegura la seqüencialitat.

Prevenció *Deadlock*

Eliminar esperes cícliques: tercer protocol.

c Ordenar recursos amb 2PLP

Regla: si la transacció ja ha bloquejat un element, **no pot demanar el bloqueig d'elements anteriors** en l'ordre.

Exemple: `Q1 < Q2 < Q3 < Q4`

Q1	**Q2**	**Q3**	**Q4**

`T` ja té `Q2`: **pot demanar** `Q3` o `Q4`, però **no** `Q1`.

Requisit

Cal conèixer els recursos a bloquejar **abans d'executar**.

Avantatge

Implica **pocs canvis** respecte del protocol 2PLP.

Seqüencialitat per conflictes, gràcies al 2PLP, i lliure de deadlock. La recuperabilitat depèn de la variant de 2PLP (estricte o rigorós).

Prevenció *Deadlock*

Per prevenir *deadlocks*, el sistema pot **expropiar bloqueigs i fer retrocedir transaccions**.

Expropiació	**Marca temporal `TS(T)`**
Treure el bloqueig d'un element a la transacció que el té i fer-la **retrocedir** fins a un punt no conflictiu.	Cada transacció rep una marca que **decideix si espera o retrocedeix**. Si retrocedeix, **conserva la marca**.

Dos protocols basats en la marca temporal

d — **Esperar-Morir** (*Wait-Die*)
Sense expropiació

e — **Ferir-Esperar** (*Wound-Wait*)
Amb expropiació

Esquemes de prevenció de deadlock i d'inanició aplicats sobre el 2PLP: la seqüencialitat **per conflictes** i la recuperabilitat vénen del 2PLP de base.

Prevenció *Deadlock*

d) Esperar-Morir (*Wait-Die*), sense expropiació.
`Ti` vol bloquejar `Q`, però el té `Tj`.

Ti més antiga: `TS(Ti) < TS(Tj)`	**Ti més recent: `TS(Tj) < TS(Ti)`**
`Ti` **espera**.	`Ti` **retrocedeix (mor)** i es reinicia més tard amb la mateixa marca `TS(Ti)`.

Exemple: `TS(T22) < TS(T23) < TS(T24)`

`T22` demana `Q` a `T23` → `T22` **espera** (és més antiga).

`T24` demana `Q` a `T23` → `T24` **retrocedeix** (és més recent).

Prevenció *Deadlock*

e) Ferir-Esperar (*Wound-Wait*), amb expropiació.
`Ti` vol bloquejar `Q`, però el té `Tj`.

Ti més antiga: `TS(Ti) < TS(Tj)`	**Ti més recent: `TS(Tj) < TS(Ti)`**
Expropiació: `Tj` **retrocedeix** o es cancel·la i es reinicia més tard amb la mateixa marca `TS(Tj)`.	`Ti` **espera**.

Exemple: `TS(T22) < TS(T23) < TS(T24)`

`T22` demana `Q` a `T23` → `T22` **bloqueja** `Q` i `T23` **retrocedeix** o es cancel·la.

`T24` demana `Q` a `T23` → `T24` **espera** (és més recent).

Prevenció *Deadlock*

Diferències entre els protocols *Esperar-Morir* i *Ferir-Esperar*:

Esperar-Morir

Transacció antiga

- Espera que la més recent alliberi el bloqueig: com més antiga, més espera.

Retrocessos

- Si Ti retrocedeix per un element Q de Tj, ho torna a intentar més tard. Si Tj continua amb Q, Ti pot retrocedir diverses vegades.

Ferir-Esperar

Transacció antiga

- No espera la transacció nova.

Retrocessos

- Ti passa a l'espera: hi ha menys retrocessos.

Gestió de deadlocks

01 Prevenció

02 **Detecció i recuperació**

Detecció i recuperació de *Deadlock*

Periòdicament s'aplica un **algorisme de detecció**. Si troba *deadlock*, cal **recuperar-se**.

Per a la detecció, el sistema ha de:

1 Mantenir informació

Sobre l'assignació d'elements de dades a les transaccions.

2 Detectar

Definir un algorisme que comprovi si hi ha deadlock.

3 Recuperar

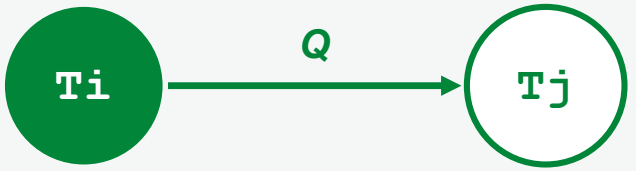
Sortir del deadlock si l'algorisme el detecta.

Detecció *Deadlock*: graf d'espera

Graf d'espera: graf dirigit *G = (V, A)* que mostra qui espera qui.

V	**Vèrtexs**: les transaccions.
A	**Arcs**: `Ti → Tj` vol dir que `Ti` espera que `Tj` alliberi un element que té.

Exemple



`Ti` espera *Q*, que té `Tj`

El graf s'actualitza amb cada bloqueig i desbloqueig

`bloqueja_*()` → afegir arc	`desbloqueja()` → eliminar arc
`Ti` demana *Q*, que té `Tj`: s'**afegeix** l'arc `Ti → Tj`.	`Tj` allibera el recurs que espera `Ti`: s'**elimina** l'arc `Ti → Tj`.

Detecció *Deadlock*

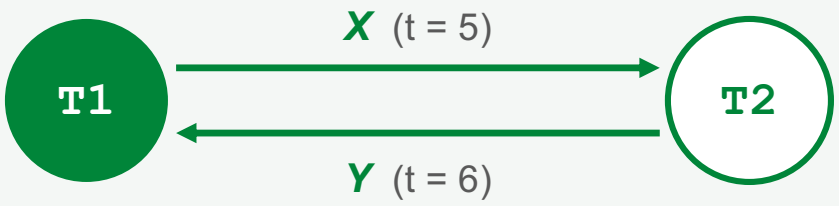
Hi ha ***deadlock*** si i només si el graf d'espera **té un cicle**.

Detecció	Un **algorisme de cerca de cicles** recorre el graf. Les transaccions del cicle (dues o més) són les implicades en el *deadlock*.

Exemple amb dues transaccions

t	*T1*	*T2*
1	`bloqueja_s(Y)`	
2	`r(Y)`	
3		`bloqueja_x(X)`
4		`r(X)`
5	`bloqueja_x(X)`	
6		`bloqueja_x(Y)`

Graf d'espera



Cicle `T1` → `T2` → `T1`: ***deadlock*** a t = 6

Detecció *Deadlock*

Quan s'aplica l'**algorisme de cerca de cicles**?

1 Interval fix	2 Deadlocks freqüents	3 Cas pitjor
Es defineix cada quant **temps** s'executa l'algorisme.	Si hi ha *deadlocks* sovint, s'executa **més sovint**.	Detecció a **cada petició de bloqueig**: molt ***overhead***.

Menys execucions → **Més execucions**

> **Compromís:** com més sovint s'executa, abans es detecta el *deadlock*, però més cost té per al sistema.

Recuperació de *Deadlock*

Cal recuperar quan s'ha detectat el *deadlock*.

Solució: retrocedir una o més transaccions per trencar el *deadlock*.

Cal fer tres accions:

1 Seleccionar víctima	2 Retrocés	3 Inanició
Quines transaccions cal retrocedir.	Fins on cal retrocedir.	Evitar que es triï sempre la mateixa víctima.

Recuperació de *Deadlock*

Seleccionar víctima: quina o quines transaccions cal fer retrocedir.

Factors per decidir-ho:

1. **Feina feta i pendent**
   Què ha calculat la transacció i què li falta.

2. **Elements utilitzats**
   Nombre d'elements de dades que ja fa servir.

3. **Elements pendents**
   Nombre d'elements de dades que necessita per acabar.

4. **Transaccions afectades**
   Nombre de transaccions involucrades.

Recuperació de *Deadlock*

Retrocés: determinar fins on cal retrocedir. Opcions:

Retrocés total	Avortar la transacció i reiniciar-la.
Retrocés parcial	Retrocedir fins que es desbloquegin els elements de dades en conflicte. Cal calcular quins bloqueigs alliberar.

Inanició: risc de triar sempre la mateixa víctima.

Solució	Triar cada transacció com a víctima un nombre finit de vegades: el nombre de retrocessos entra com a factor de cost.

Continguts

01 Introducció

02 Protocols basats en bloqueigs

03 Gestió de deadlocks

04 **Altres problemes de concurrència**

Problemes de control de concurrència

Altres problemes que el control de concurrència ha de tenir en compte:

Inserció i eliminació	**També generen problemes**, no només les operacions de llegir i escriure ja vistes.
Transaccions interactives	Llegeixen i escriuen en un dispositiu interactiu (el monitor). L'usuari pot introduir un valor basat en **dades de pantalla no confirmades**.
Panys	Bloqueigs de **curta durada**, fora del control del bloqueig en dues fases.

Gràcies

Dubtes i preguntes

Gestió i Administració de Bases de Dades. Grau en Enginyeria Informàtica

Oriol Ramos Terrades

Luis Santiago Luévano

Departament de Ciències de la Computació

Escola d'Enginyeria UAB

DC UAB